

Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение высшего образования
«Московский государственный технический университет имени Н.Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ «Информатика и системы управления»

КАФЕДРА «Программное обеспечение ЭВМ и информационные технологии»

Лабораторная работа № 2
по дисциплине «Анализ алгоритмов»

Тема Сравнительный анализ алгоритмов умножения матриц

Студент Крылов В.С.

Группа ИУ7-52Б

Преподаватели Волкова Л.Л.

Преподаватели Строганов Д.В.

Москва, 2026

СОДЕРЖАНИЕ

ВВЕДЕНИЕ **3**

1 Аналитическая часть **4**

1.1 Постановка задачи 4

1.2 Вывод 4

2 Конструкторская часть **5**

2.1 Требования к программному обеспечению 5

2.2 Разработка алгоритмов 5

2.2.1 Стандартный алгоритм умножения матриц 6

2.2.2 Алгоритм Винограда 8

2.2.3 Оптимизированный алгоритм Винограда 13

2.3 Вывод 18

3 Технологическая часть **19**

3.1 Средства разработки 19

3.2 Реализация алгоритмов 19

3.3 Тестирование реализаций 22

3.4 Вывод 22

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ **23**

ВВЕДЕНИЕ

Целью лабораторной работы является выполнение сравнительного анализа алгоритмов умножения матриц по трудоёмкости.

Задачи работы:

— описать решаемую задачу согласно варианту;

— разработать алгоритмы решения задачи согласно варианту: стандартный алгоритм умножения матриц, алгоритм Винограда и оптимизированный по варианту алгоритм Винограда;

— описать разработанные алгоритмы схемами алгоритмов;

— описать требования к программному обеспечению;

— описать средства реализации алгоритмов;

— реализовать разработанные алгоритмы;

— выполнить тестирование реализации алгоритмов;

— описать модель вычислений для оценки трудоёмкости алгоритмов и выполнить теоретический расчёт трудоёмкости;

— описать инструменты замера процессорного времени выполнения реализаций алгоритмов;

— выполнить массированные замеры процессорного времени выполнения реализаций алгоритмов умножения матриц в зависимости от линейного размера квадратных матриц, отдельно для чётных и нечётных размеров;

— выполнить сравнительный анализ алгоритмов по трудоёмкости и по данным выполненных замеров.

1 Аналитическая часть

1.1 Постановка задачи

Даны матрицы A размера $M \times N$ и B размера $N \times Q$, содержащие целочисленные элементы. Требуется вычислить результирующую матрицу $C = A \times B$ размера $M \times Q$, элементы которой определяются соотношением:

$$C_{i,j} = \sum_{k=0}^{N-1} A_{i,k} \cdot B_{k,j}, \quad i = 0, \ldots, M-1, \quad j = 0, \ldots, Q-1. \tag{1.1}$$

Операция умножения матриц определена тогда и только тогда, когда количество столбцов матрицы A совпадает с количеством строк матрицы B.

Для решения поставленной задачи требуется разработать и программно реализовать следующие алгоритмы:

— стандартный алгоритм умножения матриц;

— базовый алгоритм Винограда;

— оптимизированный по варианту алгоритм Винограда.

Требуется сравнить разработанные реализации между собой по теоретической трудоёмкости и по данным замеров процессорного времени их выполнения на квадратных матрицах ($M = N = Q$), с раздельным рассмотрением для чётных и нечётных линейных размеров матриц.

Входные данные: линейные размеры матриц $M, N, Q \in \mathbb{N}$ (для квадратных матриц — линейный размер N) и целочисленные элементы матриц $A_{i,k}, B_{k,j} \in \mathbb{Z}$. Выходные данные: матрица C размера $M \times Q$ ($N \times N$ для квадратных матриц), элементы которой $C_{i,j} \in \mathbb{Z}$.

1.2 Вывод

В результате аналитической части была формализована задача умножения матриц, определены форматы входных и выходных данных, а также сформирован перечень алгоритмов, подлежащих разработке, реализации и сравнительному анализу.

2 Конструкторская часть

2.1 Требования к программному обеспечению

К разрабатываемому программному обеспечению предъявляются следующие требования:

1. ввод линейных размеров матриц M, N, Q (натуральные числа);
2. ввод целочисленных элементов матриц A и B вручную с клавиатуры либо автоматическая случайная генерация элементов в заданном числовом диапазоне;
3. вывод текущих матриц A и B в консоль в форматированном виде;
4. циклический вывод интерактивного меню с возможностью выбора и выполнения следующих операций:
   - 4.1. вычисление произведения матриц с использованием стандартного алгоритма;
   - 4.2. вычисление произведения матриц с использованием базового алгоритма Винограда;
   - 4.3. вычисление произведения матриц с использованием оптимизированного по варианту алгоритма Винограда;
   - 4.4. перегенерация исходных матриц случайными значениями;
   - 4.5. повторный ввод размерностей и элементов матриц пользователем;
   - 4.6. вывод текущих матриц на экран;
   - 4.7. запуск серии массированных замеров процессорного времени выполнения всех трёх алгоритмов для квадратных матриц чётных и нечётных размеров с выгрузкой результатов в CSV-файлы;
   - 4.8. завершение работы программы.

2.2 Разработка алгоритмов

Для каждого из разрабатываемых алгоритмов умножения определены:

— A — входная матрица размера $M \times N$ с целочисленными элементами;

— B — входная матрица размера $N \times Q$ с целочисленными элементами;

— C — результирующая матрица размера $M \times Q$, элементы которой перед началом вычислений инициализируются нулями;

— M, N, Q — линейные размеры перемножаемых матриц.

Для базового и оптимизированного алгоритмов Винограда дополнительно определены:

— row — вспомогательный вектор длины M для промежуточных сумм парных произведений строк матрицы A;

— col — вспомогательный вектор длины Q для промежуточных сумм парных произведений столбцов матрицы B;

Для оптимизированного алгоритма Винограда дополнительно определены:

— $half_n$ — кэшированная граница диапазонов циклов $\lfloor N/2 \rfloor$;

— two_k — локальная переменная, хранящая вычисленный сдвигом индекс $k \ll 1$ $(2k)$.

2.2.1 Стандартный алгоритм умножения матриц

В стандартном алгоритме производится вычисление каждого элемента $C_{i,j}$ результирующей матрицы по математическому определению произведения матриц[// ссылка]:

$$C_{i,j} = \sum_{k=0}^{N-1} A_{i,k} \cdot B_{k,j}, \quad i = 0, \ldots, M-1, \quad j = 0, \ldots, Q-1. \tag{2.1}$$

Алгоритм состоит из трёх вложенных циклов: внешний цикл организует проход по строкам $i \in [0, M-1]$, средний — по столбцам $j \in [0, Q-1]$, а внутренний выполняет накопление суммы произведений по индексу $k \in [0, N-1]$[// ссылка]. Схема стандартного алгоритма умножения матриц приведена на рисунке 2.1.

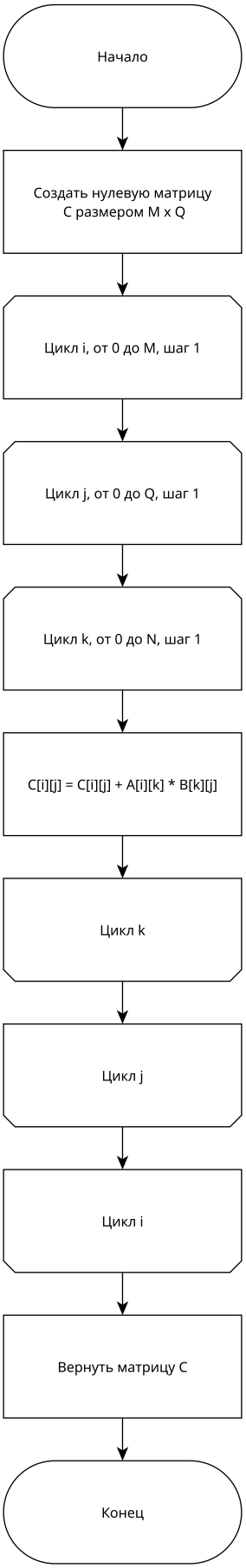


Рисунок 2.1 — Схема стандартного алгоритма умножения матриц

2.2.2 Алгоритм Винограда

В алгоритме Винограда уменьшается общее число скалярных умножений элементов матриц ценой увеличения количества операций сложения[// ссылка]. Вычисление произведения основано на предварительном расчёте двух одномерных массивов, зависящих только от строк матрицы A и столбцов матрицы B соответственно:

$$row_i = \sum_{k=0}^{\lfloor N/2 \rfloor - 1} A_{i,2k} \cdot A_{i,2k+1}, \quad i = 0, \ldots, M-1, \tag{2.2}$$

$$col_j = \sum_{k=0}^{\lfloor N/2 \rfloor - 1} B_{2k,j} \cdot B_{2k+1,j}, \quad j = 0, \ldots, Q-1. \tag{2.3}$$

После вычисления массивов row и col каждый элемент матрицы результата инициализируется значением $-row_i - col_j$, после чего в основном цикле суммируются произведения парных сумм элементов:

$$C_{i,j} = -row_i - col_j + \sum_{k=0}^{\lfloor N/2 \rfloor - 1} (A_{i,2k} + B_{2k+1,j}) \cdot (A_{i,2k+1} + B_{2k,j}). \tag{2.4}$$

Если общий размер N является нечётным числом, последний непарный элемент строки и столбца учитывается отдельной коррекцией:

$$C_{i,j} = C_{i,j} + A_{i,N-1} \cdot B_{N-1,j}. \tag{2.5}$$

Схема базового алгоритма Винограда приведена на рисунках 2.2 - 2.5.

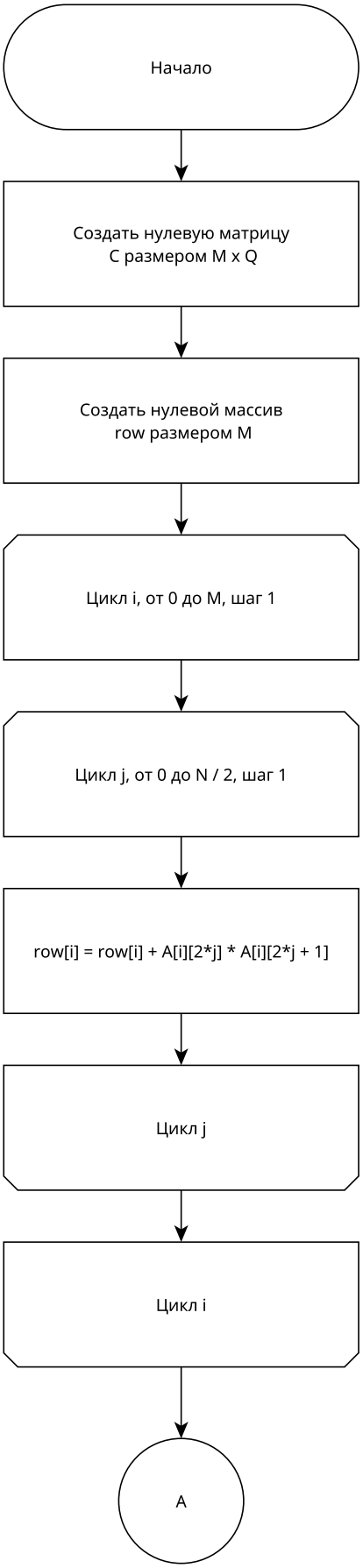


Рисунок 2.2 — Схема базового алгоритма Винограда

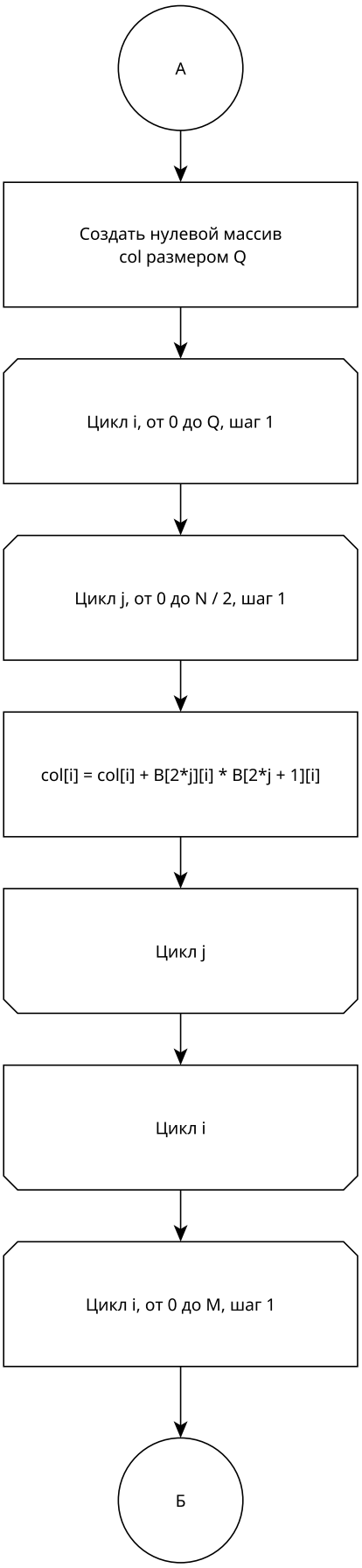


Рисунок 2.3 — Схема продолжения базового алгоритма Винограда

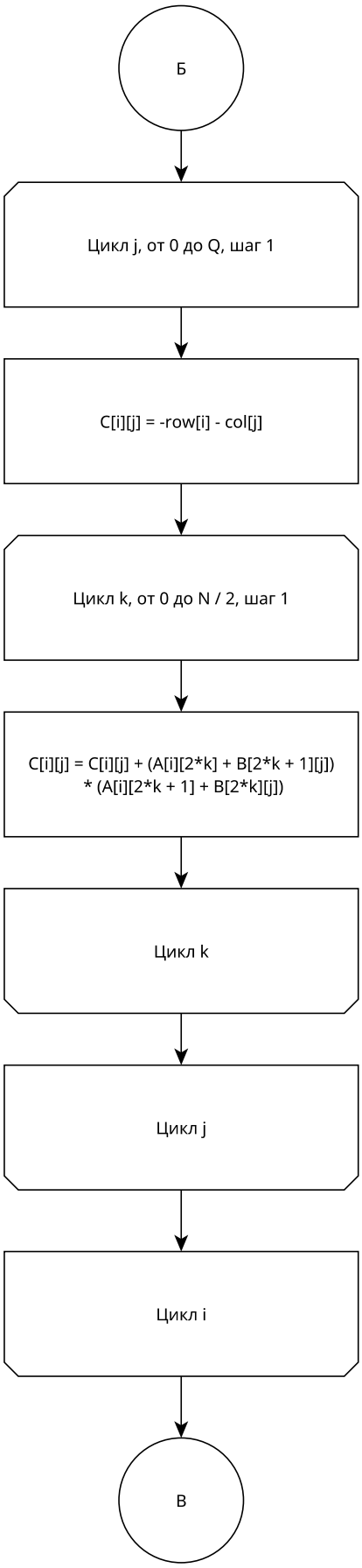


Рисунок 2.4 — Схема продолжения базового алгоритма Винограда

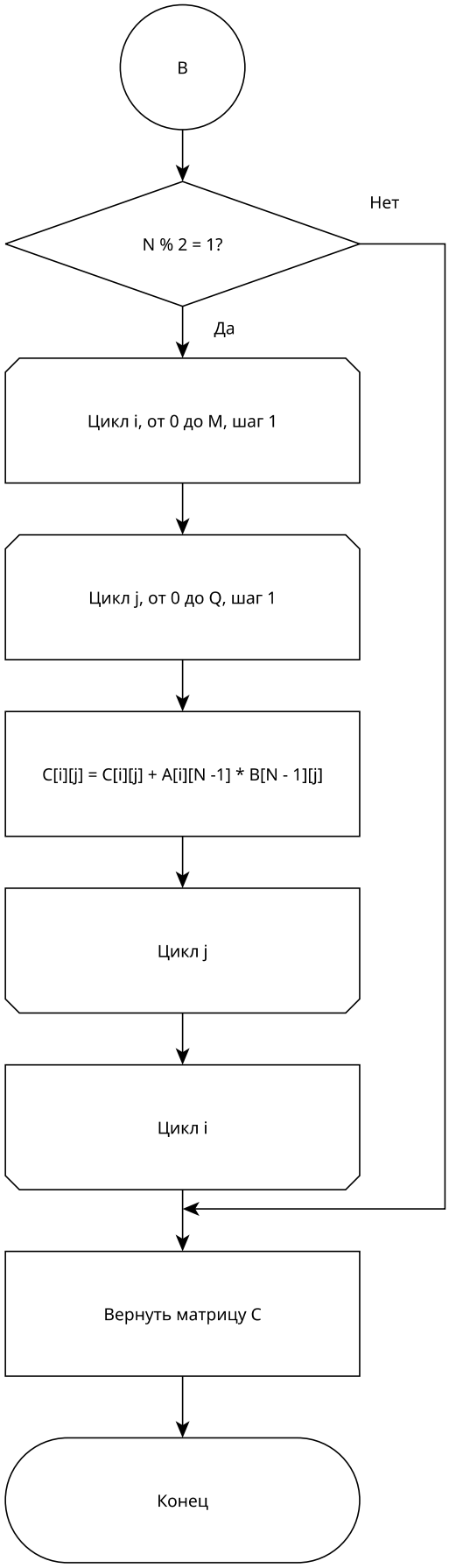


Рисунок 2.5 — Схема продолжения базового алгоритма Винограда

2.2.3 Оптимизированный алгоритм Винограда

В соответствии с индивидуальным вариантом к алгоритму Винограда применены следующие две оптимизации:

1. замена умножения $2 \cdot k$ при индексации побитовым сдвигом влево на один разряд: $k \ll 1$. Значение вычисляется один раз в начале каждой итерации цикла по k, сохраняется в локальную переменную two_k и затем повторно используется во всех обращениях к элементам текущей итерации;
2. предварительное вычисление (кэширование) значения верхней границы циклов $half_n = \lfloor N/2 \rfloor$ в отдельную константу до входа в циклы вместо многократного повторения целочисленного деления $N/2$ при каждой проверке условия продолжения цикла.

Применение данных оптимизаций не изменяет результирующие значения матрицы C и общее количество операций над её элементами, однако сокращает накладные расходы на целочисленную арифметику внутри циклов. Схемы оптимизированного алгоритма Винограда приведена на рисунках 2.6 - 2.10.

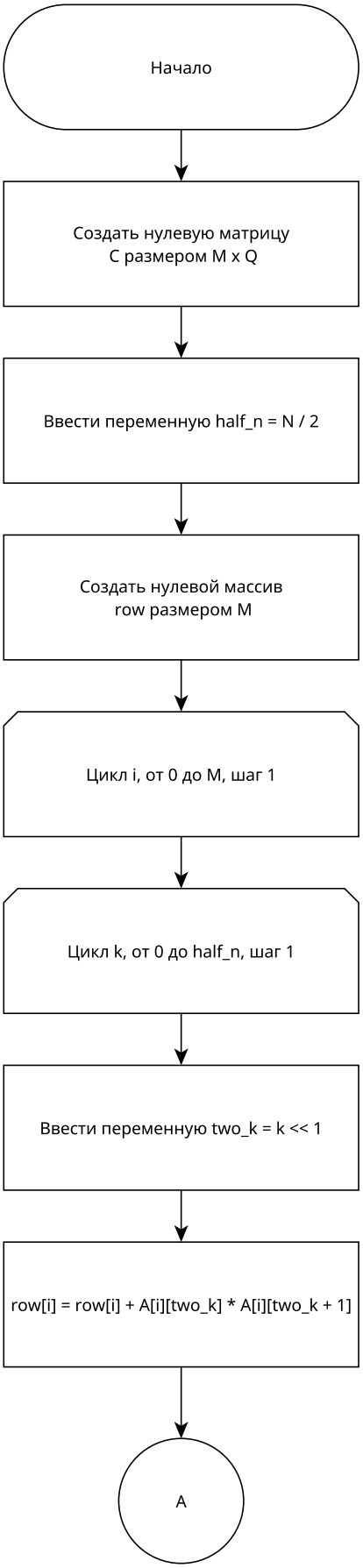


Рисунок 2.6 — Схема оптимизированного алгоритма Винограда

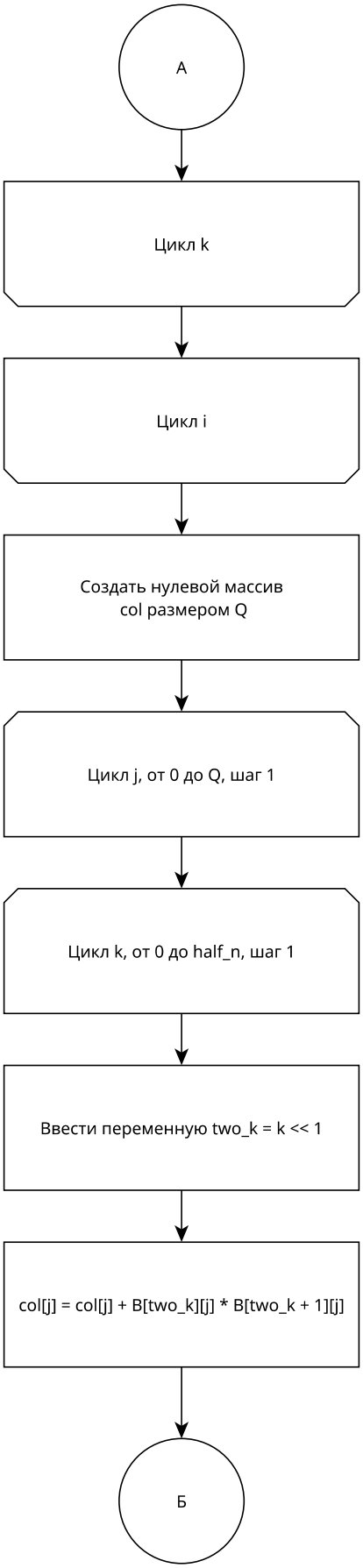


Рисунок 2.7 — Схема продолжения оптимизированного алгоритма Винограда

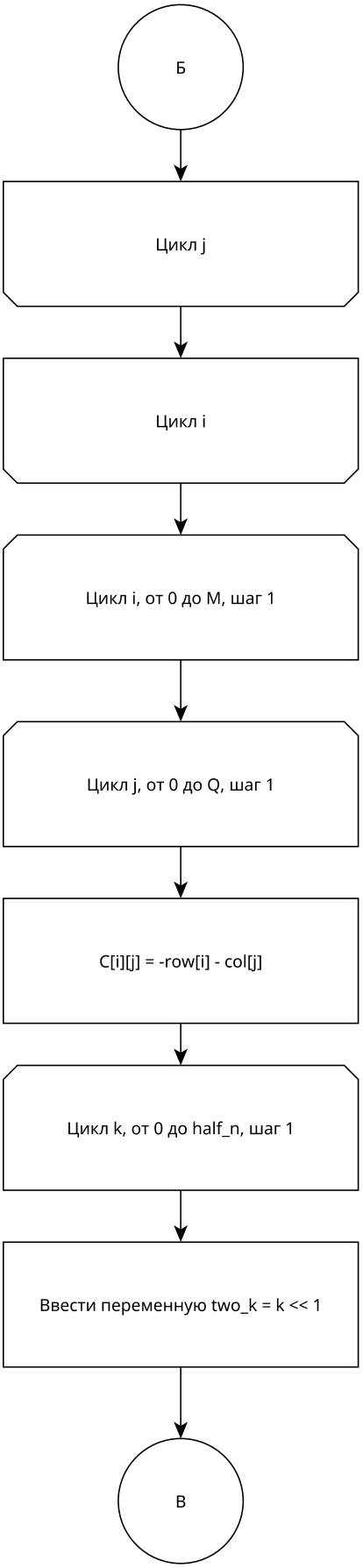


Рисунок 2.8 — Схема продолжения оптимизированного алгоритма Винограда

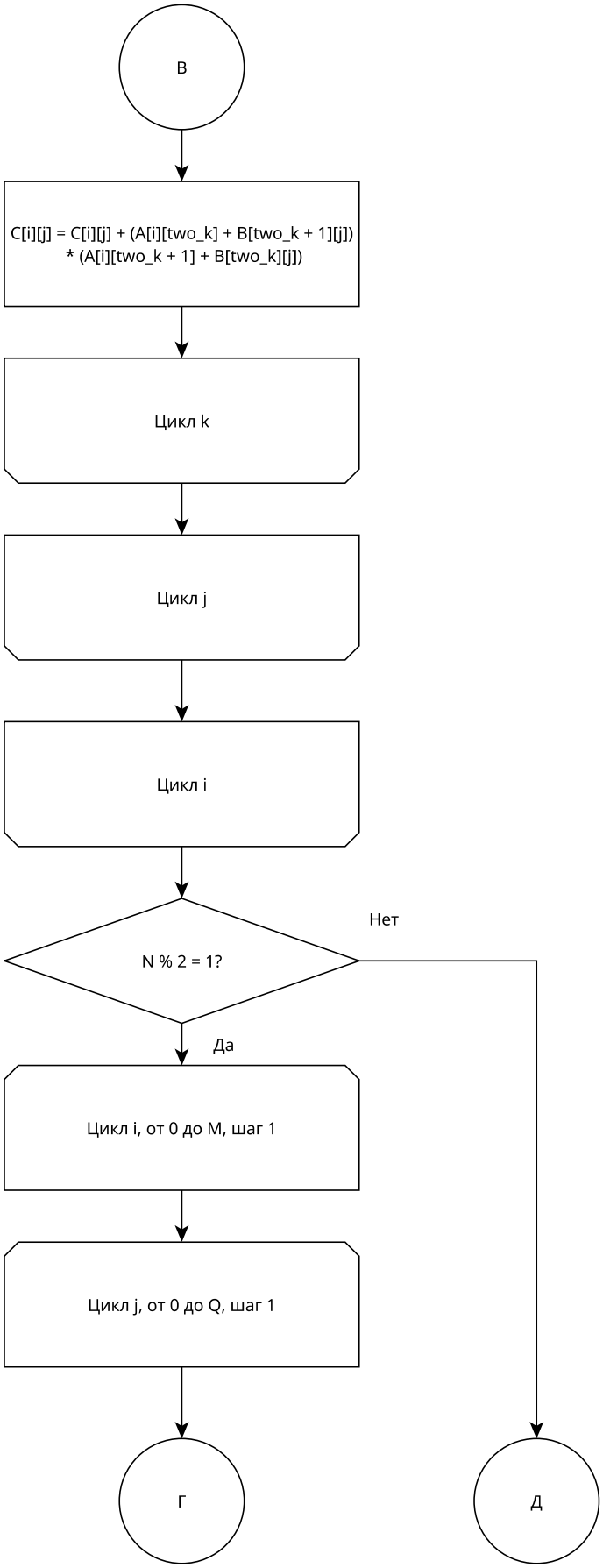


Рисунок 2.9 — Схема продолжения оптимизированного алгоритма Винограда

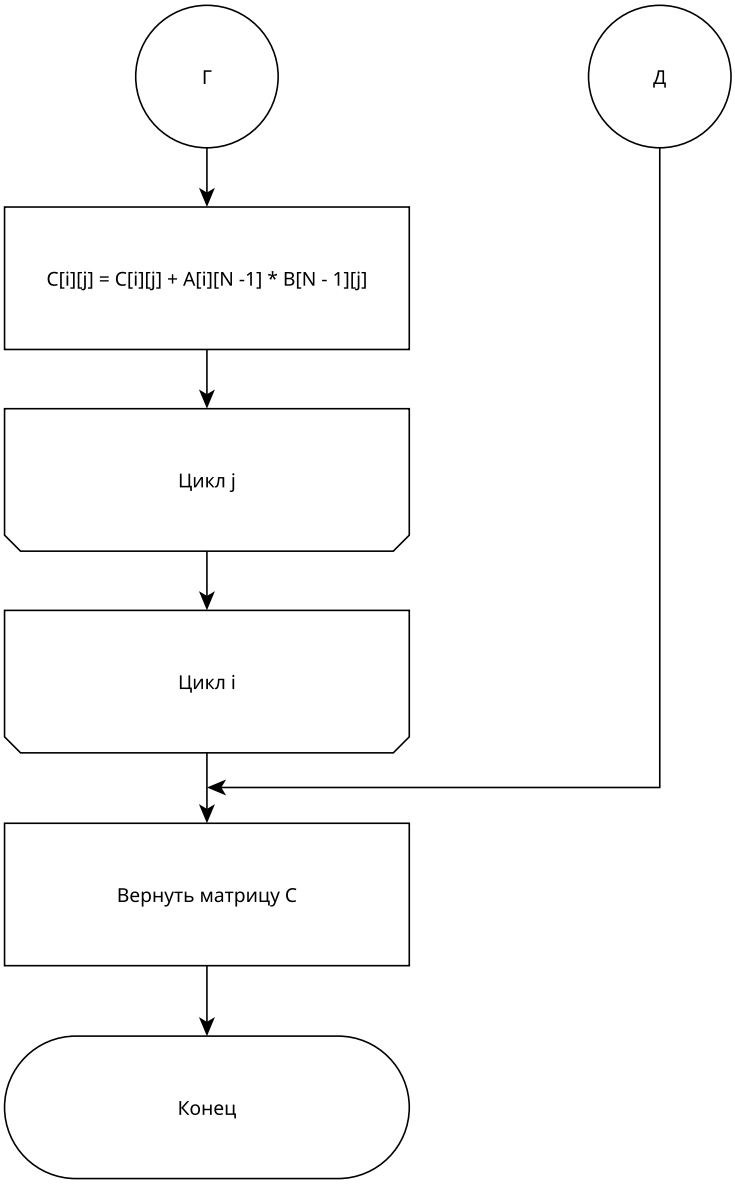


Рисунок 2.10 — Схема продолжения оптимизированного алгоритма Винограда

2.3 Вывод

В результате выполнения конструкторской части были формализованы требования к разрабатываемому программному обеспечению, описаны математические основы и спецификации входных и выходных данных для стандартного алгоритма умножения матриц, базового алгоритма Винограда и его оптимизированной модификации, а также представлены схемы данных алгоритмов.

3 Технологическая часть

3.1 Средства разработки

В качестве языка программирования со статической типизацией был выбран язык C++ (стандарт C++20) [1]. В процессе разработки использовались следующие средства стандартной библиотеки языка:

— контейнер `std::vector<std::vector<int>>` — для динамического представления целочисленных матриц;

— контейнер `std::vector<int>` — для хранения вспомогательных массивов row и col;

— функция `clock()` из заголовочного файла `<ctime>` — для замера процессорного времени работы исследуемых алгоритмов;

Сборка программы осуществлялась компилятором GCC с флагами компиляции:

```
-std=c++20 -Wall -Wextra -Wpedantic -O0
```

Флаг оптимизации `-O0` задан принудительно, чтобы компилятор не выполнял автоматическую оптимизацию.

3.2 Реализация алгоритмов

На рисунке 3.1 представлена программная реализация стандартного алгоритма умножения матриц.

```
Matrix multiply_standard(const Matrix &A, const Matrix &B)
{
    const size_t M = A.size();
    const size_t N = A[0].size();
    const size_t Q = B[0].size();

    Matrix C = create_matrix(M, Q, 0);

    for (size_t i = 0; i < M; ++i)
        for (size_t j = 0; j < Q; ++j)
            for (size_t k = 0; k < N; ++k)
                C[i][j] += A[i][k] * B[k][j];

    return C;
}
```

Рисунок 3.1 — Стандартный алгоритм умножения матриц

На рисунке 3.2 представлена реализация базового алгоритма Винограда.

```
Matrix multiply_winograd(const Matrix &A, const Matrix &B)
{
    const size_t M = A.size();
    const size_t N = A[0].size();
    const size_t Q = B[0].size();
    Matrix C = create_matrix(M, Q, 0);

    vector<int> row(M, 0);
    for (size_t i = 0; i < M; ++i)
        for (size_t j = 0; j < N / 2; ++j)
            row[i] += A[i][2 * j] * A[i][2 * j + 1];

    vector<int> col(Q, 0);
    for (size_t i = 0; i < Q; ++i)
        for (size_t j = 0; j < N / 2; ++j)
            col[i] += B[2 * j][i] * B[2 * j + 1][i];

    for (size_t i = 0; i < M; ++i)
    {
        for (size_t j = 0; j < Q; ++j)
        {
            C[i][j] = -row[i] - col[j];
            for (size_t k = 0; k < N / 2; ++k)
                C[i][j] += (A[i][2 * k] + B[2 * k + 1][j]) * (A[i][2 * k + 1] +
                    B[2 * k][j]);
        }
    }

    if (N % 2 == 1)
        for (size_t i = 0; i < M; ++i)
            for (size_t j = 0; j < Q; ++j)
                C[i][j] += A[i][N - 1] * B[N - 1][j];

    return C;
}
```

Рисунок 3.2 — Базовый алгоритм Винограда

На рисунке 3.3 представлена реализация оптимизированного алгоритма Винограда.

```
Matrix multiply_winograd_opt(const Matrix &A, const Matrix &B)
{
    const size_t M = A.size();
    const size_t N = A[0].size();
    const size_t Q = B[0].size();
    Matrix C = create_matrix(M, Q, 0);
    const size_t half_n = N / 2;

    vector<int> row(M, 0);
    for (size_t i = 0; i < M; ++i)
    {
        for (size_t k = 0; k < half_n; ++k)
        {
            const size_t two_k = k << 1;
            row[i] += A[i][two_k] * A[i][two_k + 1];
        }
    }

    vector<int> col(Q, 0);
    for (size_t j = 0; j < Q; ++j)
    {
        for (size_t k = 0; k < half_n; ++k)
        {
            const size_t two_k = k << 1;
            col[j] += B[two_k][j] * B[two_k + 1][j];
        }
    }

    for (size_t i = 0; i < M; ++i)
    {
        for (size_t j = 0; j < Q; ++j)
        {
            C[i][j] = -row[i] - col[j];
            for (size_t k = 0; k < half_n; ++k)
            {
                const size_t two_k = k << 1;
                C[i][j] += (A[i][two_k] + B[two_k + 1][j]) * (A[i][two_k + 1] +
                    B[two_k][j]);
            }
        }
    }

    if (N % 2 == 1)
        for (size_t i = 0; i < M; ++i)
            for (size_t j = 0; j < Q; ++j)
                C[i][j] += A[i][N - 1] * B[N - 1][j];

    return C;
}
```

Рисунок 3.3 — Оптимизированный алгоритм Винограда

3.3 Тестирование реализаций

В таблице 3.1 представлены результаты функционального тестирования разработанных реализаций алгоритмов умножения матриц.

Таблица 3.1 — Функциональные тесты

№	Входные данные		Выходные данные		Комментарий
	Матрица A	**Матрица** B	**Ожид.**	**Получ.**	
1	$\begin{pmatrix}3\end{pmatrix}$	$\begin{pmatrix}-4\end{pmatrix}$	$\begin{pmatrix}-12\end{pmatrix}$	$\begin{pmatrix}-12\end{pmatrix}$	Матрицы размера 1×1
2	$\begin{pmatrix}1&2&3\\4&5&6\\7&8&9\end{pmatrix}$	$\begin{pmatrix}1&0&0\\0&1&0\\0&0&1\end{pmatrix}$	$\begin{pmatrix}1&2&3\\4&5&6\\7&8&9\end{pmatrix}$	$\begin{pmatrix}1&2&3\\4&5&6\\7&8&9\end{pmatrix}$	Умножение на единичную матрицу $E_{3\times3}$
3	$\begin{pmatrix}1&2\\3&4\end{pmatrix}$	$\begin{pmatrix}0&0\\0&0\end{pmatrix}$	$\begin{pmatrix}0&0\\0&0\end{pmatrix}$	$\begin{pmatrix}0&0\\0&0\end{pmatrix}$	Умножение на нулевую матрицу
4	$\begin{pmatrix}1&2\\3&4\end{pmatrix}$	$\begin{pmatrix}5&6\\7&8\end{pmatrix}$	$\begin{pmatrix}19&22\\43&50\end{pmatrix}$	$\begin{pmatrix}19&22\\43&50\end{pmatrix}$	Квадратные матрицы 2×2, чётный размер
5	$\begin{pmatrix}1&2&3\\4&5&6\\7&8&9\end{pmatrix}$	$\begin{pmatrix}9&8&7\\6&5&4\\3&2&1\end{pmatrix}$	$\begin{pmatrix}30&24&18\\84&69&54\\138&114&90\end{pmatrix}$	$\begin{pmatrix}30&24&18\\84&69&54\\138&114&90\end{pmatrix}$	Квадратные матрицы 3×3, нечётный размер
6	$\begin{pmatrix}1&2&3\\4&5&6\end{pmatrix}$	$\begin{pmatrix}7&8\\9&10\\11&12\end{pmatrix}$	$\begin{pmatrix}58&64\\139&154\end{pmatrix}$	$\begin{pmatrix}58&64\\139&154\end{pmatrix}$	Прямоугольные матрицы 2×3 и 3×2
7	$\begin{pmatrix}1&2\\3&4\\5&6\end{pmatrix}$	$\begin{pmatrix}1&2&3&4\\5&6&7&8\end{pmatrix}$	$\begin{pmatrix}11&14&17&20\\23&30&37&44\\35&46&57&68\end{pmatrix}$	$\begin{pmatrix}11&14&17&20\\23&30&37&44\\35&46&57&68\end{pmatrix}$	Прямоугольные матрицы 3×2 и 2×4
8	$\begin{pmatrix}1&2&3&4\end{pmatrix}$	$\begin{pmatrix}1\\2\\3\\4\end{pmatrix}$	$\begin{pmatrix}30\end{pmatrix}$	$\begin{pmatrix}30\end{pmatrix}$	Вектор-строка на вектор-столбец
9	$\begin{pmatrix}1\\2\\3\end{pmatrix}$	$\begin{pmatrix}4&5\end{pmatrix}$	$\begin{pmatrix}4&5\\8&10\\12&15\end{pmatrix}$	$\begin{pmatrix}4&5\\8&10\\12&15\end{pmatrix}$	Вектор-столбец на вектор-строку

Все тесты были пройдены успешно без расхождений в результатах.

3.4 Вывод

В результате выполнения технологической части были обоснованы средства реализации, приведены листинги исходного кода стандартного, базового и оптимизированного алгоритмов Винограда на языке C++, а также разработана система тестов, подтвердившая корректность всех разработанных реализаций.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. ISO/IEC 14882:2020. Programming languages — C++. — International Organization for Standardization, 2020.

2. Winograd, S. A New Algorithm for Inner Product / S. Winograd // IEEE Transactions on Computers. — 1968. — Vol. C-17, no. 7. — P. 693–694.

3. Кормен, Т. Алгоритмы: построение и анализ : пер. с англ. / Т. Кормен, Ч. Лейзерсон, Р. Ривест, К. Штайн. — 3-е изд. — М. : ООО «И.Д. Вильямс», 2013. — 1328 с.